

Guide for the creation and use of ONNX file

A ONNX file (Open Neural Network Exchange) is a type of file for AI models that offers the ability to move models from a specific framework like PyTorch or Tensorflow to other ecosystems. This brings versatility and opens possibilities for the use of AI, as not all ecosystems are capable of reading or using AI models in the most used frameworks like the ones mentioned before.

In the LabVIEW projects made for this investigation, we used a ONNX file for predictions within LabVIEW in two of them. For this reason, this guide was made with the intention of explaining how to transfer a PyTorch model to ONNX, showing the function and different variables that need to be set up in order to have a functioning ONNX file. It is important to know what each variable does, as the definition of each could change how you program your Python and LabVIEW programs that use this ONNX file.

The following code shows the function used to export the model trained in PyTorch into a ONNX model, using the onnx library that is inside the PyTorch libraries, but also download the onnx and onnxruntime libraries as you may need them.

Export code

```
dummy_input = torch.randn(1, 2000, 2).to(device) #Adjust shape to match model's input
onnx_filename = "zeus_laser_tracker.onnx"

torch.onnx.export(
    PyTorch_model,          # The trained model instance
    dummy_input,            # A dummy input tensor with the correct shape
    onnx_filename,          # Output file name
    export_params=True,     # Store the trained parameter weights inside the file
    opset_version=17,       # Modern, highly stable ONNX operator version
    do_constant_folding=True, # Optimizes the network math by pre-calculating constants
    input_names=['input'],  # Clean names for your input layer
    output_names=['predicted_xy'], # Clean names for your output layer
    dynamic_axes={          # Allows you to use flexible batch sizes in production if needed
        'input_timeline': {0: 'batch_size'},
        'predicted_xy': {0: 'batch_size'}
    }
)
```

Code explanation

The function to call is `torch.onnx.export`, as said the exporting function is found inside the PyTorch libraries. Before using the function, it would be recommended to create the filename for your new onnx file, and a dummy input that has the same shape as your input, so that your ONNX export also knows the shape of your input. Next, a quick description of each variable of the function is given below.

- `PyTorch_model`: Here you put your PyTorch trained model.
- `dummy_input`: This is a dummy variable created with the exact shape of the input for the model (in this case a tensor of shape (1, 2000, 2))
- `onnx_filename`: This is the name of the file you wish to give to your ONNX model.
- `export_params`: when given True, this ensures that the weights of the model are stored inside the ONNX file.
- `Opset_version`: This is the version of your ONNX file, make sure to keep it in a newer version.
- `Do_constant_folding`: When True, your static weight values will be simplified, making the model lighter and faster.
- `input_names`: This is the name you will give your input when predicting. Later in the document it will be shown how this part is used. It is usually better to just name your input 'input'.
- `output_names`: Here we establish the name of our output, in this case named `predicted_xy`, since our output are X and Y coordinates.
- `Dynamic_axes`: with this variable we can define if any of our input shapes can be dynamic. For example, a dynamic shape is given into our dim 0, our batch size, so that we can have a batch size bigger than 1, instead of only a static 1. This is not necessary but was used in case of a bigger batch size.

Next, the code used for opening the ONNX file, predicting, and receiving the information in LabVIEW will be shown and the parts related to the use of the ONNX file will be explained.

Code for using ONNX file in LabVIEW

```
import numpy as np
import onnxruntime as ort
import os
import time

# Get the directory where this script is located
script_dir = os.path.dirname(os.path.abspath(__file__))

# Initialize the ONNX session
onnx_path = os.path.join(script_dir, "zeus_laser_tracker_fixed.onnx")
session = ort.InferenceSession(onnx_path)

# Load the scaling parameters
scaling_path = os.path.join(script_dir, "scaling_params.npz")
scaling_data = np.load(scaling_path)
mean = scaling_data['mean'] # [mean_x, mean_y]
std = scaling_data['std'] # [std_x, std_y]

def predict_coordinates(lv_array_2d):
    start_time = time.perf_counter()

    # Reconstruct the 2D matrix layout from LabVIEW
    raw_flat = np.array(lv_array_2d, dtype=np.float32).flatten()
    reconstructed_2d = raw_flat.reshape(-1, 2)
    current_rows = reconstructed_2d.shape[0]

    # Dynamic padding
    repeats = int(np.ceil(2000 / current_rows))
    padded_data = np.tile(reconstructed_2d, (repeats, 1))
    truncated_data = padded_data[:2000, :]

    # STANDARDIZE THE INCOMING DATA
    standardized_data = (truncated_data - mean) / std

    # Reshape to 3D Tensor format: (1, 2000, 2)
    input_tensor = np.expand_dims(standardized_data, axis=0).astype(np.float32)

    # Execute ONNX Inference
    outputs = session.run(['predicted_xy'], {'input': input_tensor})

    # UN-STANDARDIZE THE PREDICTION
    scaled_pred = outputs[0][0]
    real_x = float((scaled_pred[0] * std[0]) + mean[0])
    real_y = float((scaled_pred[1] * std[1]) + mean[1])
```

```

end_time = time.perf_counter()
elapsed_ms = (end_time - start_time) * 1000.0 # Convert to milliseconds

# Format as a 2D nested list containing: [[X, Y, Time_ms]]
prediction_2d = [[real_x, real_y, elapsed_ms]]

return prediction_2d

```

Code Explanation

The general workflow of the code is the next:

1. Setting up paths for script and ONNX file, and opening a ONNX session
2. Opening files with mean and std and saving them in variables
3. Reading input form LabVIEW and reshaping it
4. Normalize data received
5. Reshape array into tensor (format for ONNX file)
6. Run model to predict X and Y coordinates
7. PostProcessing
8. Return data to LabVIEW

We will focus on points 1, 5 and 6 as this are the ones that have relation to the ONNX file

- 1: The two following lines are what opens the ONNX file, and its own session:

```

onnx_path = os.path.join(script_dir, "zeus_laser_tracker_fixed.onnx")
session = ort.InferenceSession(onnx_path)

```

The path for the file is opened using the os library, and it is in hte same directory as the script, to simplify code. As for the ONNX session, you need to have and use the onnxruntime library (downloadable with pip) and use the function InferenceSession with the path of your ONNX file as input.

- 5: We receive form LabVIEW a 2D array with X and Y coordinates, but the ONNX model has as an input a 3D array, so we reshape the normalized array into the necessary shape (1, 2000, 2). The first dimension is the amount of batches, the second the rows, and third the columns. This redimensioning is done in the next line of code:

```

input_tensor = np.expand_dims(standardized_data, axis=0).astype(np.float32)

```

- 6: Finally, we use the function .run to do the inference. For this line we need to indicate the output and input, in this case ['predicted,xy'] and {'input':

input_tensor} being the correct names for both output and input. Remember that you need to put the exact name you programmed when you saved the ONNX file in the export.

```
outputs = session.run(['predicted_xy'], {'input': input_tensor})
```

This is everything you need to know about how to export a PyTorch model into ONNX, and how to open it and run inferences with it. Remember to be careful and make sure that you name and call your input and output correctly, and to give the correct shape for the input always.

References

ONNX export example:

https://docs.pytorch.org/tutorials/beginner/onnx/export_simple_model_to_onnx_tutorial.html